Kathmandu University

Department of Computer Science and Engineering Dhulikhel, Kavre

A Project Report

on

"ChronoSync: Intelligent Alarm & Timer Management System"

[Code No :COMP 202]

(For partial fulfillment of II Year/I Semester in Computer Engineering)

Submitted by:
Anusha Khatri (61)

Submitted to:

Mr. Sagar Acharya

Department of Computer Science and Engineering

Submission Date:2026/02/21

Abstract

ChronoSync: Intelligent Alarm & Timer Management System is a modern desktop-based time management application designed to enhance productivity and daily scheduling efficiency. In today's fast-paced environment, individuals often struggle with managing time effectively due to busy schedules, academic workloads, and professional commitments. ChronoSync addresses this challenge by providing a unified platform for alarm scheduling, countdown timers, Pomodoro sessions, and recurring reminders. ChronoSync features a modern graphical interface that ensures smooth navigation and real-time updates. The system emphasizes offline usability, making it accessible without internet dependency. By integrating essential time management tools into a single application, ChronoSync transforms traditional alarm systems into a comprehensive productivity assistant.

Future enhancements may include cloud synchronization, mobile integration, and smart notifications based on user habits.

Keywords: Time management, alarm system, countdown timer, Pomodoro technique, C, C++, Qt framework, productivity tools.

Table of contents

Abstract..... 2

List of Figures..... 5

Chapter 1 Introduction..... 6

1.1 Background..... 6

1.2 Objectives.....6

1.3 Motivation and Significance..... 7

Chapter 2 Related Works..... 7

Chapter 3 Design and Implementation..... 10

3.1 Implementation..... 10

3.2 System Requirement Specifications.....15

3.2.1 Software Specifications.....15

3.2.2 Hardware Specifications.....16

Chapter 4 Discussion on the achievements..... 17

4.1 Features:.....19

Chapter 5 Conclusion and Recommendation.....20

5.1 Limitations..... 21

5.2 Future Enhancement..... 22

References.....23

APPENDIX..... 24

LIST OF FIGURES

Figure 2.1 Alarmy user interface 8

Figure 2.2 Google clock user interface 9

Figure 2.3 SleepCycle user interface 9

Figure 3.1 ChronoSync System Flowchart 12

Figure 3.2 Use Case Diagram 13

Fig A.1: ChronoSync's Main Page 24

Chapter 1 Introduction

1.1 Background

Effective time management is crucial for academic success, professional productivity, and personal well-being. With increasing workloads and digital distractions, individuals often rely on multiple tools such as alarm clocks, timers, and scheduling applications. However, these tools are typically fragmented and lack integration, leading to inefficiencies.

ChronoSync was developed as a unified solution that combines alarm scheduling, countdown timers, and productivity-focused features into a single desktop application. Built using the Qt framework with C++, the system offers a lightweight yet powerful platform for managing daily routines.

The name “ChronoSync” represents synchronization of time-based activities (“Chrono” meaning time). The system aims to help users stay organized, punctual, and productive through intelligent reminders and scheduling tools.

1.2 Objectives

The main objectives of ChronoSync are:

- To develop a graphical user interface for setting alarms and timers.
- To implement multiple alarm scheduling with snooze and recurring features.
- To provide productivity tools such as Pomodoro timer.
- To ensure reliable alarm notifications with sound alerts.
- To create an offline desktop solution for time management.

1.3 Motivation and Significance

The motivation for selecting the topic “ChronoSync” arises from the increasing difficulty individuals face in managing time effectively in a fast-paced digital environment. Students and professionals often struggle with maintaining routines, meeting deadlines, and balancing multiple tasks simultaneously. Traditional alarm clocks and many existing alarm applications provide only basic alert functionality without supporting productivity-focused features such as structured timers or intelligent reminders.

Existing systems often lack customization, offline accessibility, or integration between alarms and productivity tools. ChronoSync differs by providing a unified platform that combines alarm scheduling and focus-based timing techniques within a lightweight Qt-based interface. This approach makes the system suitable for academic and professional use without requiring internet connectivity.

Chapter 2 Related Works

Over the past few years, various alarm clock and productivity management applications have been developed to help users organize their daily schedules, manage time effectively, and improve productivity. These systems not only provide basic alarm functionality but also include advanced features such as countdown timers, Pomodoro sessions, recurring alarms, reminders, and analytics of time usage. While many applications share the common goal of improving time management, they differ in terms of interface design, customization, offline functionality, and integration of productivity tools. Some of the notable systems related to our proposed system are discussed below:

2.1 Alarmy

Alarmy is a popular alarm clock application designed to ensure users wake up on time by requiring them to complete specific tasks—such as solving math problems, shaking the phone, or taking a picture—to dismiss the alarm. It includes features like loud ringing sounds, snooze options, and customizable alarm tones. Alarmy focuses strongly on wake-up effectiveness rather than broader productivity management. However, it does not provide integrated countdown timers or Pomodoro-based study sessions within the same system. In contrast, our proposed system, ChronoSync, combines intelligent alarms with countdown timers and productivity tools in a

unified interface, making it more suitable for students and professionals who need both waking and task management support.

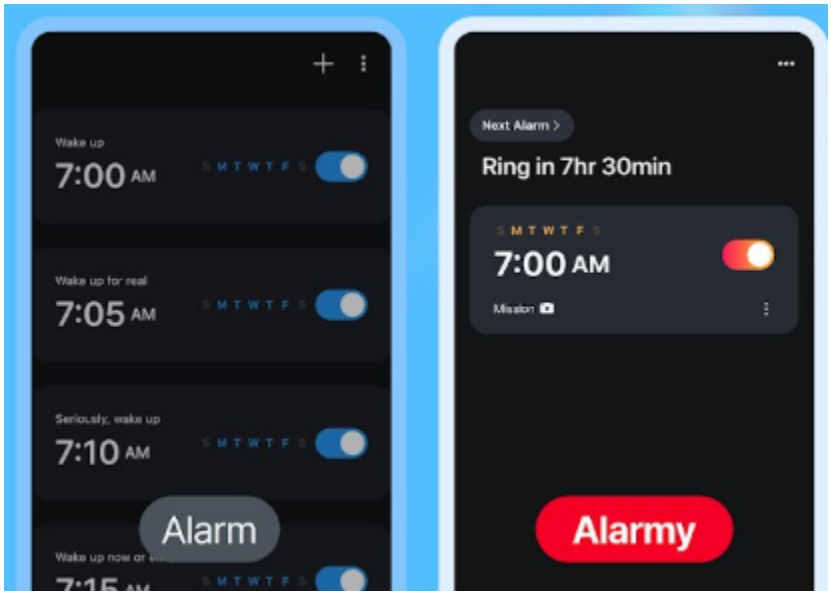

Figure 2.1: Alarmy user interface

2.2 Google Clock

Google Clock is a standard clock application that provides alarms, timers, stopwatch, and world clock features with a clean and simple interface. It supports recurring alarms, gradual volume increase, and integration with calendars. While it is efficient for everyday use, it lacks advanced productivity features such as Pomodoro timers, motivational alarm messages, and customizable scheduling patterns. ChronoSync, on the other hand, extends beyond basic alarm functionality by incorporating Pomodoro sessions, recurring productivity timers, and personalized alarm messages, enabling users to manage both sleep schedules and focused work sessions within one system.

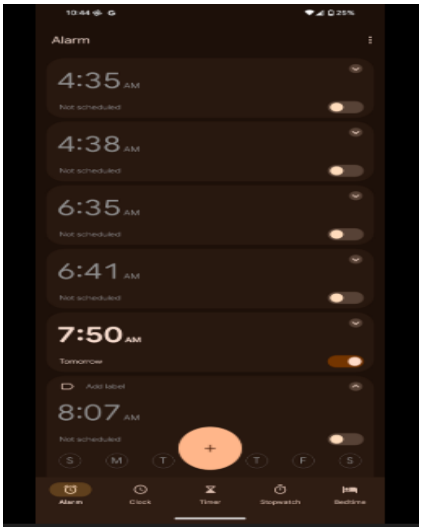

Figure 2.2: Google clock user interface

2.3 Sleep Cycle

Sleep Cycle is an advanced alarm system that analyzes sleep patterns and wakes users during their lightest sleep phase within a selected time window. It uses sound analysis and sensors to track sleep behavior and provides insights into sleep quality. Although it offers detailed analytics and smart wake-up features, it primarily focuses on sleep health rather than daily task productivity. ChronoSync differs by targeting productivity-oriented users, offering features such as countdown timers, Pomodoro cycles, recurring alarms, and customizable notifications in addition to standard alarm functions. The system highlights the importance of smart wake-up technology but does not include task-oriented time management tools.

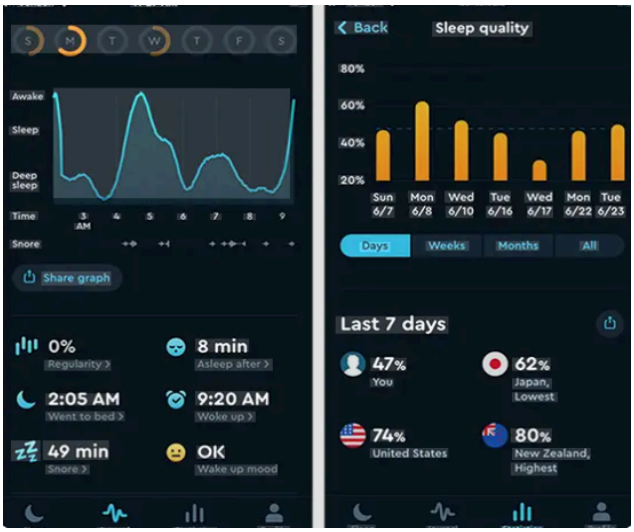

Figure 2.3: SleepCycle user interface

Chapter 3: Design and Implementation

3.1 Implementation

This section describes the systematic methodology and sequential procedures followed in the development of ChronoSync, our intelligent alarm and timer management desktop application. The development process followed a structured lifecycle beginning from concept formulation and requirement analysis to system design, development, testing, and release. The goal was to create a user-friendly productivity tool that integrates alarm scheduling, countdown timers, and Pomodoro-based focus sessions within a single modern interface.

3.1.1 Planning

The initial phase involved extensive discussions to define the objectives and major functionalities of the system. The primary aim was to develop a smart time-management solution that helps users maintain punctuality, improve productivity, and manage daily schedules efficiently. I identified the need for a system that allows users to plan, focus, and perform their daily tasks through intelligent alarms and timers.

Following brainstorming sessions, I researched the technologies required for implementation, including the Qt framework for GUI development, C, C++ for core logic, and multimedia libraries for alarm sound handling.

3.1.2 Requirement analysis

Based on all the knowledge and insights gained from the planning phase, we identified the following functional requirements for our system:

Functional Requirements

- Alarm Management: Create, edit, delete, and schedule alarms with customizable labels and tones.
- Countdown Timer: Provide countdown functionality for tasks, events, and deadlines.
- Pomodoro Timer: Support productivity sessions with work and break intervals.
- Recurring Alarms: Enable daily or weekly repeating alarms.
- Snooze Feature: Allow temporary postponement of alarms.
- Notifications and Messages: Display alert messages when alarms or timers trigger.
- Real-Time Clock Display: Show current time dynamically on the interface.

Non-Functional Requirements

- User-friendly interface with clear navigation
- System responsiveness and reliability
- Offline functionality without internet dependency
- Accurate timing and synchronization
- Minimal resource consumption

3.1.3 System Design

During the design phase, conceptual ideas were transformed into a technical blueprint. UI prototypes were created using Figma to visualize layout and navigation flow. These designs were later implemented using the Qt framework.

The system architecture was designed to ensure seamless interaction between the graphical interface and backend logic. Flowcharts were prepared to map the operational logic of alarms, timers, and productivity sessions. A Use Case Diagram illustrated user interactions with core features such as setting alarms, starting timers, and managing schedules.

The system was designed modularly with separate components for alarm management, timer operations, notification handling, and user interface control. This modular approach improved maintainability and scalability. Here the above the flow chart and user case diagram illustrates how chronosync works with proper explanation with optimization features and user details how users can implement it further as shown in the figure below:

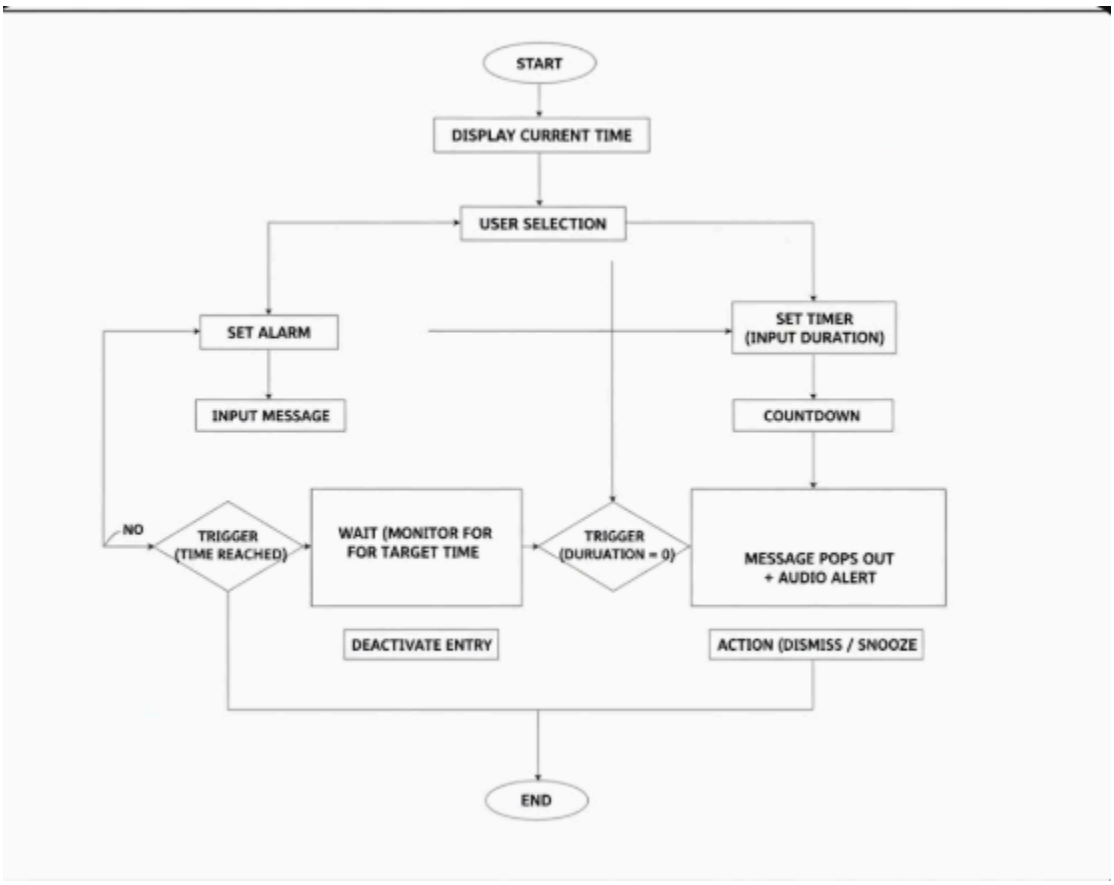

Fig 3.1: ChronoSync System Flowchart

The flowchart represents the logical flow where users launch the application, set alarms or timers, manage schedules, and receive notifications and messages when time events occur.

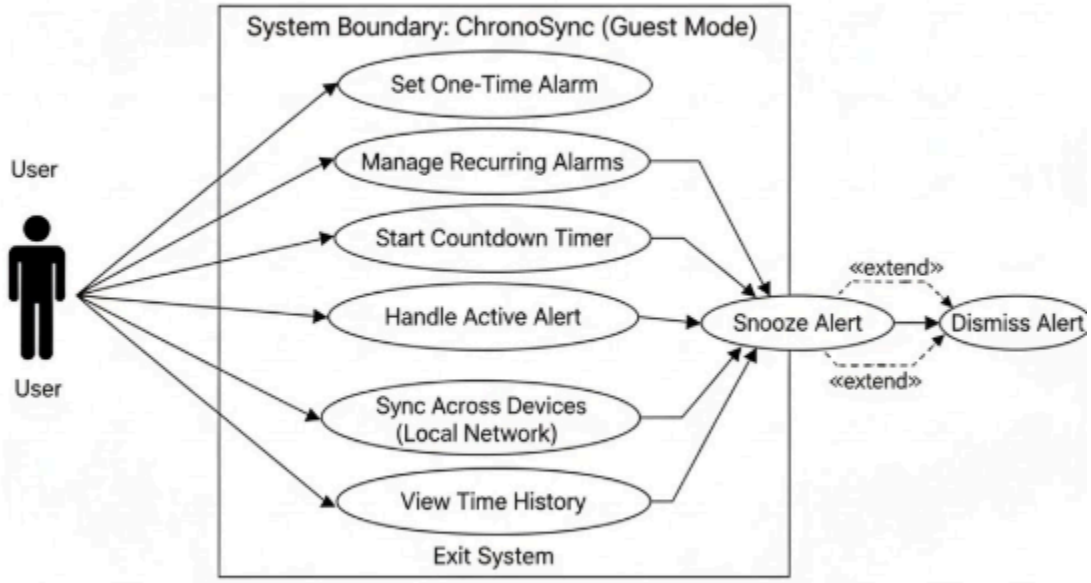

Fig 3.2: Use Case Diagram

The use case diagram illustrates interactions between the user and ChronoSync, including creating alarms, starting countdowns, using Pomodoro sessions, and exiting the system.

3.1.4 Development

With the design finalized, development of ChronoSync proceeded using C++ and the Qt framework, with particular emphasis on applying appropriate Data Structures and Algorithms (DSA) to ensure efficiency, accuracy, and responsiveness of the system. Different modules of the application utilized specific data structures based on their functional requirements.

Use of Data Structures and Algorithms in ChronoSync:

● **Alarm Scheduling** — Priority Queue (Min Heap)
A priority queue implemented using a min-heap structure was used to manage multiple alarms efficiently. Since alarms must trigger based on the earliest scheduled time, the alarm with the minimum time value is always kept at the top. This allows the system to quickly determine which alarm should ring next without scanning all alarms.

● **Alarm Storage and Management** — Dynamic Array (Vector/List)
All created alarms were stored in a dynamic array structure to allow flexible insertion, deletion, and modification. This supports CRUD operations such as adding new alarms, editing existing ones, and deleting unwanted alarms while maintaining efficient memory usage.

● **Recurring Alarm Handling** — Circular Queue
Recurring alarms (daily/weekly) were managed using a circular queue concept, allowing alarms to repeat automatically after execution without recreating them manually. Once an alarm triggers, it is reinserted at the appropriate future time slot

● **Countdown Timer** — Queue Structure

Countdown timers were implemented using a queue-based logic where time decrements sequentially. This structure ensures first-in-first-out execution for multiple timers running simultaneously.

3.1.5 Testing and Debugging

The testing phase was critical to ensuring the accuracy, performance, and reliability of Chronosync. This included:

Testing ensured system accuracy, performance, and reliability.

- Unit Testing: Individual modules such as alarm triggering and timer countdown were tested separately.
- Integration Testing: Verified interaction between UI components and backend logic.
- Input Validation: Checked invalid inputs like incorrect time formats.
- Timing Accuracy Testing: Compared system timing with actual clock time to ensure precision.
- System Testing: Tested complete workflow from setting alarms to receiving notifications.

Bugs were resolved through debugging and logic refinement, improving performance and stability.

3.1.6 Project Release and Enhancement

ChronoSync was developed primarily for academic purposes and released through GitHub for review and further improvement. The repository includes source code, documentation, and instructions for building and running the application locally.

Future enhancements may include mobile synchronization, cloud backup, task scheduling integration, and analytics for productivity tracking.

3.2 System Requirement Specifications

This section outlines the technical requirements needed to develop and operate the system. It highlights the software and hardware setup, along with the tools used during the development of this system.

3.2.1 Software Specifications

This system was built using both frontend and backend technologies. The following frontend and backend tools used in this system is listed below:

3.2.1.1 Front End tools

Qt Framework

Qt was used to design the graphical user interface. Components such as QLabel for clock display, QPushButton for controls, QTimeEdit for setting alarms, and QTabWidget for navigation between alarm and timer sections were utilized to create an interactive interface.

3.2.1.2 Back End tools

C / C++

C and C++ were used to implement the core system logic of ChronoSync, including time calculations, alarm scheduling, timer and Pomodoro operations, and event handling.

Qt Multimedia

Qt Multimedia was utilized for playing alarm sounds, notifications, and reminders, ensuring a realistic and responsive ringing functionality.

3.2.2 Hardware Specifications

To run our system, the following hardware setup was used:

- **System Compatibility:**

The system was developed and tested on a platform capable of running C, C++ and the Qt Creator IDE, where Qt Creator was utilized to apply C++ for development without any compatibility issues.

- **Memory**

A system with 8 GB of RAM was used, ensuring smooth performance during development, UI design, and application testing.

- **Storage**

About 5 GB of storage space was used for installing Qt, saving project files, and compiled programs.

- **Display Resolution**

A screen resolution of 1920x1080 was used, which was suitable for designing and previewing the user interface in QT Designer.

- **Processor (CPU)**

An Intel Core i5 was used during development, providing efficient performance for compiling code and running the system smoothly.

3.2.3 Development tools

During the development process, we utilized essential tools that support both visual design and source code management. We used the following tools to enhance productivity, promote collaboration, and maintain consistent code quality.

Figma

Used for designing UI prototypes and layout planning before implementation.

GitHub

Used for version control, commits of projects, and project release management.

Chapter 4 Discussion on the achievements

The development of ChronoSync: Intelligent Alarm & Timer Management System was both a challenging and rewarding experience. Throughout the process, several technical and design-related challenges emerged, requiring problem-solving, logical thinking, and adaptive implementation strategies. The project aimed to create a reliable productivity tool combining alarm scheduling, countdown timers, and Pomodoro sessions within a single desktop application

One of the major challenges encountered was achieving precise timing and synchronization. Since the system depends on accurate real-time operations, ensuring that alarms ring exactly at the scheduled moment requires careful handling of system time functions and event management. Integrating multiple timing features—alarms, timers, and Pomodoro sessions—within one application also introduced concurrency issues, especially when multiple events were triggered simultaneously.

Another difficulty involved integrating the graphical user interface with backend timing logic. Synchronization problems occasionally caused delays in alarm triggering or UI freezing when heavy operations occurred. Performance optimization was necessary to ensure that the clock display updated every second without affecting other modules. Implementing recurring alarms and snooze functionality proved complex because the system needed to reschedule alarms automatically after execution. Managing these features required careful handling of time calculations and event queues.

Initially, we planned to include advanced features such as cloud synchronization and cross-device alarm sharing. However, due to time limitations and framework constraints, the system was kept as a standalone desktop application. Similarly, we intended to integrate voice notifications and AI-based scheduling suggestions, but these were simplified to standard alarm sounds and manual scheduling. Designing a modern and aesthetic user interface also posed challenges. Creating a layout that displays alarms, timers, and clock simultaneously without clutter required several iterations. Some advanced animations and visual effects were simplified due to performance considerations. During testing, platform-specific inconsistencies were observed, particularly in sound playback and window rendering across different operating systems. These issues were addressed through adjustments in multimedia handling and UI scaling.

Despite these challenges, ChronoSync successfully achieved its core objective of providing a structured and efficient time-management system. The project enhanced our understanding of real-time system design, event-driven programming, and user interface development.

4.1 Features:

The following features were implemented successfully in the project despite the complexity in the building procedure:

● Modern GUI Design

ChronoSync features a clean and intuitive graphical interface developed using Qt Widgets. The layout allows users to easily navigate between alarm, timer, and productivity sections while maintaining visual clarity.

● Alarm Management System

Users can create, edit, delete, and schedule alarms with customizable labels, tones, snooze options, and recurring settings.

● Countdown Timer

A flexible countdown timer enables users to track tasks, deadlines, and activities efficiently.

● Pomodoro Productivity Timer

The Pomodoro module supports structured work and break intervals to enhance focus and productivity.

- **Real-Time Clock Display**

The application displays the current system time dynamically, ensuring synchronization with scheduled events.

- **Message Alerts**

ChronoSync provides visual message and alerts when timers or alarms trigger.

- **Form Validation and Input Handling**

All time inputs are validated to prevent incorrect scheduling and ensure reliable system behavior.

Chapter 5 Conclusion and Recommendation

ChronoSync represents a comprehensive approach to digital time management, aligning with its core objective of helping users plan, focus, and stay punctual. The system was designed as a desktop productivity tool that integrates alarms, timers, and structured work sessions into a single platform.

Developed using C++ and the Qt framework, ChronoSync demonstrates the practical application of event-driven programming, data structures, and user interface design. The project strengthened our understanding of software development practices, real-time system implementation, and usability considerations.

Although developed primarily for academic purposes, ChronoSync has the potential to evolve into a full-featured productivity application with additional enhancements and cross-platform support.

5.1 Limitations

- **Standalone Operation**

The application operates only on a local device without cloud synchronization or multi-device access.

- **No External Integration**

ChronoSync does not integrate with calendar services, smart devices, or online scheduling platforms.

- **Manual Scheduling**

All alarms and timers must be set manually, as automatic scheduling features are not available.

- **Limited Notification Types**

Notifications are limited to visual alerts and sound playback without voice announcements or mobile alerts.

- **Platform Dependency**

Some multimedia features may behave differently depending on the operating system.

5.2 Future Enhancements

- **Cloud Synchronization**

  Enable synchronization of alarms and schedules across multiple devices.

- **Smart Scheduling**

  Incorporate AI-based suggestions for optimal work and break timings.

- **Voice Assistant Integration**

  Allow users to set alarms and timers using voice commands.

- **Calendar Integration**

  Sync with digital calendars to automatically create reminders for events.

- **Mobile Version**

  Develop a mobile application version for portability.

- **Advanced Analytics**

  Provide productivity insights based on timer usage and work patterns.

References

1. Qt Company. (n.d.). *Qt for Application Development*. Retrieved from *https://www.qt.io*
2. ChronoSync Inspiration. *Intelligent Alarm and Timer Systems Overview*. [Online]. Available: *https://www.techradar.com/best/alarm-clock-apps.* Accessed: Feb. 21, 2026.
3. Boost C++ Libraries. *Boost Chrono Library Documentation*. [Online]. Available: *https://www.boost.org/doc/libs/release/doc/html/chrono.html.* Accessed: Feb. 21, 2026.
4. Pomodoro Technique. *Official Pomodoro Technique Website*. [Online]. Available: https://francescocirillo.com/pages/pomodoro-technique. Accessed: Feb. 21, 2026.
5. Stack Overflow. *How to Implement Recurring Alarms in Qt*. [Online]. Available: *https://stackoverflow.com/questions/tagged/qt-alarm.* Accessed: Feb. 21, 2026.

Appendix

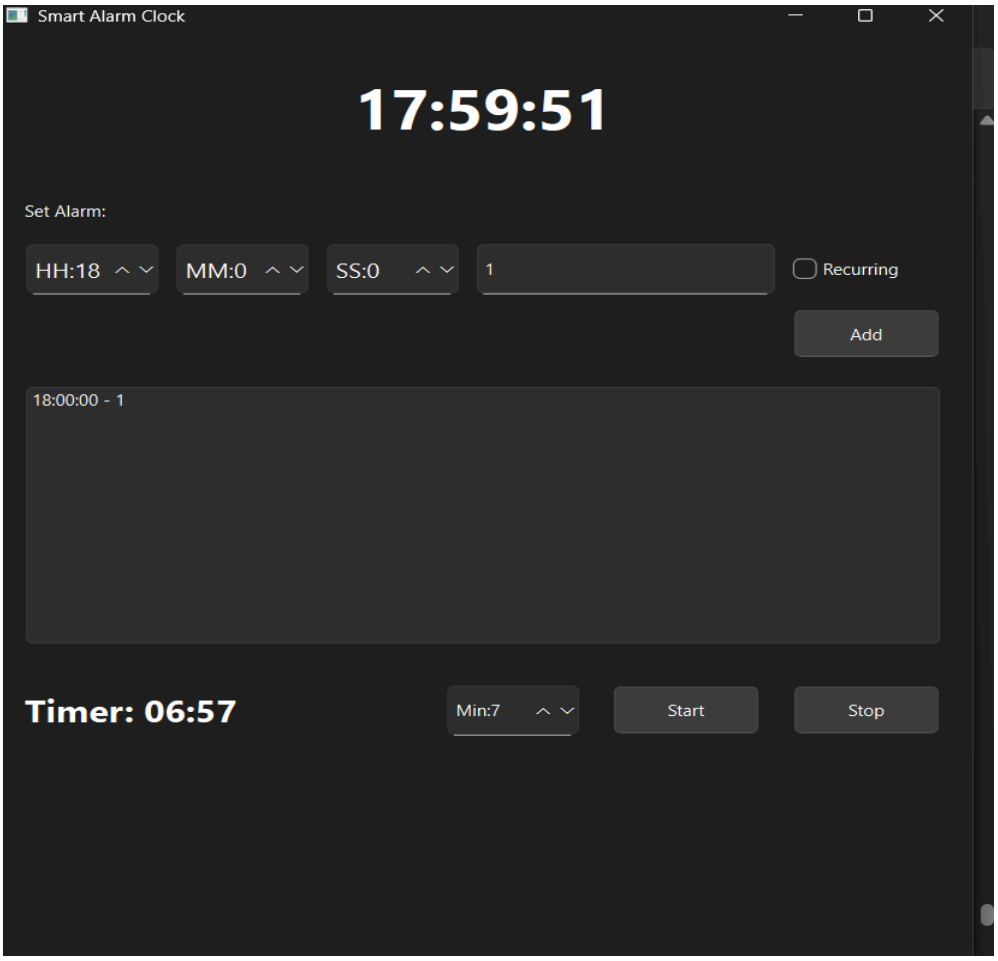

Fig A.1: ChronoSync's Main Page